BORDEAUX INP Ensc

Cyriac Dupouey

# Livrable Intermédiaire P2I - AutoEdit

**Introduction** **0**
**Semaine 1** **1**
Création de l'architecture du projet 1
Mise en place de l'environnement Python 3
Premier tests d'analyse vidéo 5
**Semaine 2** **6**
Graphique 6
Explication graphique 6
**Semaine 3** **7**
Best-of 7
**Semaine 4** **8**
1. Amélioration 1 : La Normalisation du score: 8
2. Amélioration 2 : Détecter les vrais changement de scène 8
3. Amélioration 3 : Précision du score 10
**Semaine 5** **11**
Amélioration 4 - Calculer l'intensité du moment 11
Amélioration 5 - Affichage du score d'intensité du moment 11
Amélioration 6 - Limiter la durée du best of 11
Amélioration 7 - Éviter les faux moments 12
Amélioration 8 - Ajout d'une barre de progression 12
**Objectifs à venir** **13**
**Lien de mon code source** **14**

Cyriac Dupouey

# Introduction :

L’objectif du projet est de créer un système capable d’analyser automatiquement une vidéo et d’en extraire les moments les plus dynamiques, afin de générer un résumé automatique (best-of).
L'idée est d’automatiser ce que ferait un humain :

- Regarder la vidéo
- Repérer les passages intéressants
- Couper et assembler les meilleurs moments

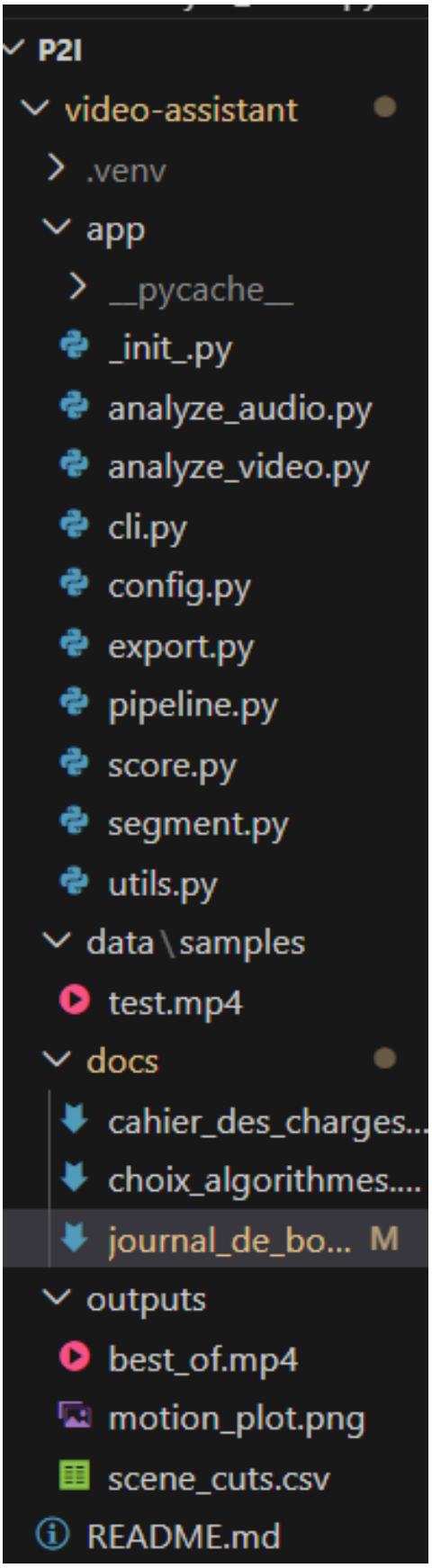
Le projet repose sur une idée simple : Plus il y’a de mouvement entre deux images consécutives, plus la scène est susceptible d'être intense ou intéressante. (bien sûr ce n’est que l’idée de base, au fur et à mesure de ma progression dans le projet, je vais implanter de nouveaux paramètres qui définissent ce qu’est une scène intéressante ou un moment fort.

On va donc exploiter dans un premier temps la différence entre frame, statistiques (moyenne + écart-type) et un filtrage temporel.

# Semaine 1

## Création de l’architecture du projet

J’ai créé l’architecture du projet dans un premier temps pour me repérer dans mon code même si j’ai commencer par tout mettre dans le fichier analyse_vidéo, je séparerai mon code plus tard (c’était plus simple pour moi au début de faire comme ca vu que mon code n’était pas trop long)


P2I
video-assistant
.venv
app
__pycache__
_init_.py
analyze_audio.py
analyze_video.py
cli.py
config.py
export.py
pipeline.py
score.py
segment.py
utils.py
data \ samples
test.mp4
docs
cahier_des_charges...
choix_algorithmes....
journal_de_bo... M
outputs
best_of.mp4
motion_plot.png
scene_cuts.csv
README.md

Cyriac Dupouey

# Mise en place de l'environnement Python

Ensuite, j'ai mis en place l'environnement python avec la commande :

- python -m venv .venv

Cela m'a permis de créer un environnement virtuel pour éviter les conflits

Puis j'ai installé les dépendances :

- pip install opencv-python numpy matplotlib

Cela m'a permis d'installer les bibliothèques dans .venv

Enfin j'ai ajouter les bibliothèques nécessaire :

```
import cv2
import numpy as np
import csv
from pathlib import Path
import matplotlib.pyplot as plt
```

1. OpenCV(cv2) : Open Source Computer Vision Library :
   Cette bibliothèque m'est très utile pour le traitement d'image, l'analyse vidéo et la vision par ordinateur. Dans mon projet elle me sert à :

   - Ouvrir une vidéo (cap = cv2.VideoCapture(video_path))
   - lire les frames (ret, frame = cap.read())
   - convertir en niveaux de gris (cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY))
   - calculer les différences entre images
   - écrire la vidéo best-of (cv2.VideoWriter(...))

2. NumPy(np)
   Cette bibliothèque est utile pour le calcul scientifique, le calcul matriciel et la manipulation de tableaux. (Les images sont des matrices de pixels)

   Calcul du mouvement : motion_score = np.sum(diff)

   Calcul de statistiques : mean_motion = np.mean(motion_array)
   std_motion = np.std(motion_array)

   Détection des frames importantes : np.where(motion_array > threshold)

   NumPy sert au calcul mathématique de l'analyse vidéo

3. CSV = Comma Separated Values
   Cette bibliothèque sert à écrire ou à lire des fichiers CSV

   Exemple de fichier :

   frame_index,time_seconds,motion_score
   10,0.33,24.5
   38,1.27,17.1

   J'exporte les coupures de scènes détectés

   writer.writerow(["frame_index", "time_seconds", "motion_score"])

   Cela permet d'analyser les résultats et de faire des statistiques

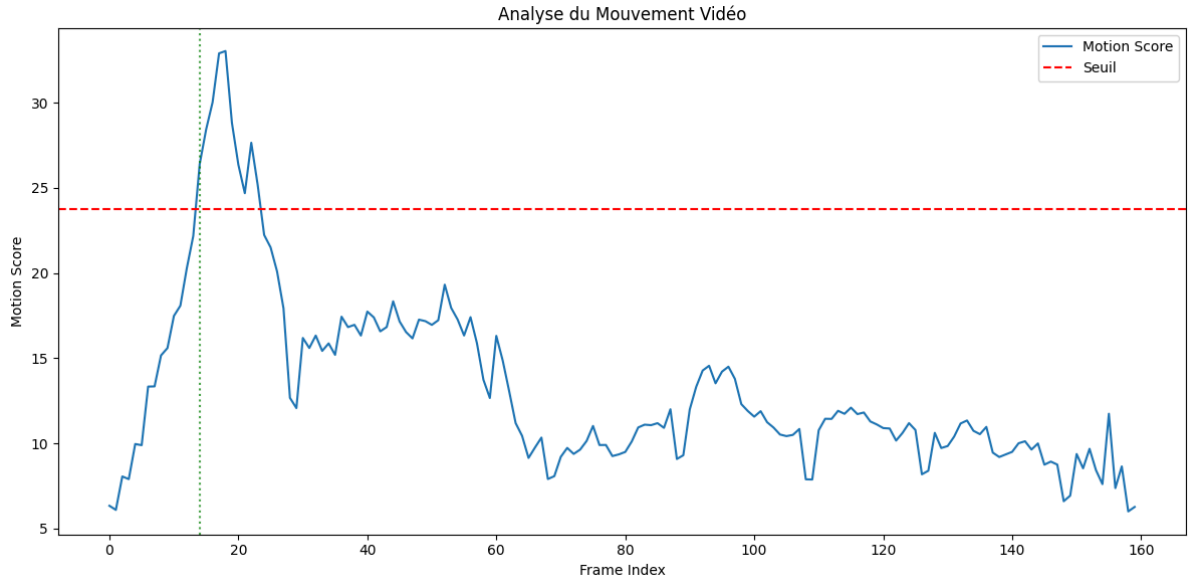
4. Pathlib(Path)
   Cette bibliothèque sert à gérer les chemins des fichiers proprement

5. Matplotlib
   Cette bibliothèque sert à visualiser les données

   Elle permet de créer :
   - graphes
   - courbes
   - histogrammes
   - visualisations scientifiques

     Dans mon sujet, je l'utilise pour visualiser le mouvement de la vidéo

     Dans mon code :
     - plt.plot(motion_array)
     - Ligne de seuil : plt.axhline(y=threshold)
     - Marquer les coupures : plt.axvline(x=cut)
     - Résultat : motion_plot.png




Ce graphe permet de mettre en évidence les moments de forte activité dans la vidéo notamment grâce au pic au dessus du seuil représenté par la ligne en pointillé rouge

## Premier tests d'analyse vidéo

J'ai réussi à analyser la vidéo en ayant les fps, les mouvements, les coupures…

Je lis la vidéo, dans analyze_video()

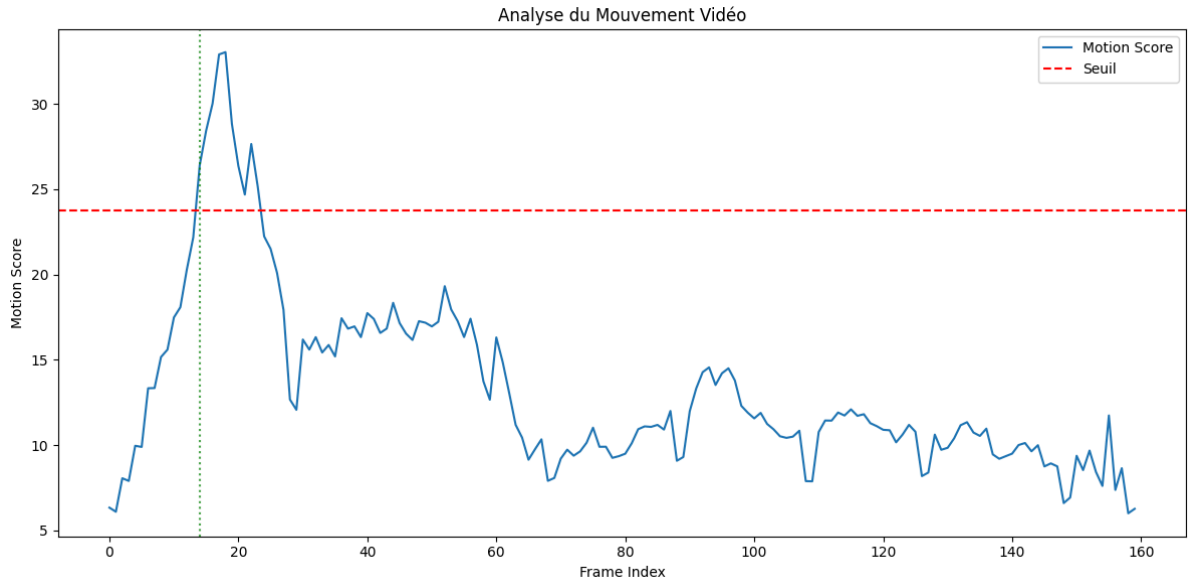
```
cap = cv2.VideoCapture(video_path)
```

On ouvre la vidéo avec OpenCV
On récupère : le nombre total de frames, le FPS (image par seconde)

Cyriac Dupouey

# Semaine 2

## Graphique




## Explication graphique

J'ai obtenu un graphique qui représente le motion_score en fonction du frame_index.
Chaque points correspond a la différence entre la frame n et la frame n-1 (Valeur faible = peu de changement visuel et valeur élevée = fort changement visuel)
La courbe rouge représente la moyenne + 2*écart-type ce qui permet de sélectionner statistiquement les anomalies.
La ligne verte correspond aux coupures détectés:
Ce sont les frames qui ont un motion_score > treshold et écart temporel > min_gap
On a donc une modélisation du mouvement par différences d'images et l'utilisation d'un seuil statistique adaptatif.
On a un pic à 2,2*10E7 = changement brutal dans l'image
Le graphique montre une distribution normale du mouvement avec :

- une anomalie externe
- un seuil adaptatif
- une détection propre et unique

# Semaine 3

## Best-of

J’essaie de générer un best of de ma vidéo a partir des paramètres étudiés dans mon fichier de code analyze_video mais sans succès.

Concernant le best-of :
On détecte les frames au dessus du seuil :

```
high_motion_frames = np.where(smoothed_motion > dynamic_threshold)[0]
```

Puis on regroupe les frames consécutives en segments :

```
moments.append((start, end))
```

On impose une durée minimale :

```
min_duration = int(fps * 1.5)
```

Pour éviter les micro-événements

Pour chaque segment :

```
cap.set(cv2.CAP_PROP_POS_FRAMES, start)
```

Ouverture de la vidéo

Puis on écrit les frames dans un nouveau fichier :

```
out.write(frame)
```

On obtient une nouvelle vidéo contenant uniquement les moments forts

Cyriac Dupouey

# Semaine 4

J'ai trouvé une solution pour ouvrir le best-of, il s'ouvre directement dans mes fichiers mais pas sur vs code mais au moins je l'ai.

## 1. Amélioration 1 : La Normalisation du score :

Actuellement mon score dépend de :
- la résolution de ma vidéo
- la luminosité

Solution :
- motion_score = np.mean(diff)

au lieu de :
- np.sum(diff)

Cela va rendre mon algorithme plus robuste , ici le score dépend de la résolution de la vidéo alors qu'avec ce changement, le score devient indépendant de la taille de la vidéo. J'utilise la moyenne des différences.

## 2. Amélioration 2 : Détecter les vrais changement de scène :

j'utilisais seulement le mouvement
Je peux ajouter :
- différence d' histogramme couleur, c'est plus fiable.

Avant je détéctais les scènes uniquement avec les mouvements mais il existe des cas ou : la caméra coupe et ou il y'a pas beaucoup de mouvement :

plan 1 : paysage calme
cut
plan 2 : autre paysage calme

L'algo ne peut pas détecter la coupure, la solution est alors de comparer les couleurs en calculant un histogramme d'image :

```
Dans la boucle :
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

J'ajoute :

hist = cv2.calcHist([gray], [0], None, [256], [0,256])
hist = cv2.normalize(hist, hist).flatten()

Puis compare avec l'histogramme précedent

Avant la boucle:

J'ajoute :

prev_hist = cv2.calcHist([prev_gray], [0], None, [256], [0,256])
prev_hist = cv2.normalize(prev_hist, prev_hist).flatten()

Dans la boucle j'ajoute :

hist_diff = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_BHATTACHARYYA)

Puis : prev_hist = hist

Cela compare la distribution des pixels

Exemple :
Image A : sombre
Image B : très lumineuse

L'histogramme est donc très différent et il y'a donc une probabilité de
cut de la scène.
```

## 3. Amélioration 3 : Précision du score

```
Au lieu de score = mouvement
je peux faire : score = 0.7 * mouvement + 0.3 * changement image

J'ai regardé sur DA vinci resolve dans les archives et c'est ce
que font les vrais systèmes vidéos

Maintenant on a :
motion_score
hist_diff

On peut les fusionner

je remplace :
motion_score = np.mean(diff)

par :

motion_score = np.mean(diff)

combined_score = motion_score + (hist_diff * 50)

puis stocke :

motion_scores.append(combined_score)

au lieu de : motion_scores.append(motion_score)

On multiplie par 50 pour amplifier l'histogramme

Changement total :
Score final = mouvement + changement de couleur

Je combine donc deux indicateurs visuels :
- La différence pixel par pixel pour mesurer le mouvement
- la distance entre histogrammes pour détécter les changements
visuels globaux
```

# Semaine 5

## Amélioration 4 - Calculer l'intensité du moment

```
je remplace : moments.append((start, prev))
par:
segment_score = np.mean(smoothed_motion[start:prev])
moments.append((start, prev, segment_score))

J'ajoute un score au segment
```

## Amélioration 5 - Affichage du score du moment

```
Après la détéction des moments, j'ajoute:
moments = sorted(moments, key=lambda x: x[2], reverse=True)
On a : X[0] = start, x[1] = end, x[2] = score


Je remplace for start, end in moments :
par :
for start, end , score in moments
J'affiche :
print(f"- {start/fps:.2f}s → {end/fps:.2f}s | score:
{score:.2f}")
```

## Amélioration 6 - Limiter la durée du best of

Mon programme exporte tous les moments
mais un best of dois faire environ 30 secondes a 1 minutes maximum, c'est ce qu'on vois en général sur les réseaux sociaux.

Avant l'export :
j'ajoute :

```
max_duration = 30 #secondes
current_duration = 0
selected_moments = []

for start, end, score in moments:

    duration = (end - start) / fps

    if current_duration + duration <= max_duration:
        selected_moments.append((start, end))
        current_duration += duration
```

je remplace : `export_best_of(video_path, moments)`
par:
`export_best_of(video_path, selected_moments)`

## Amélioration 7 - Éviter les faux moments

0.5 secondes pour le gap de la détéction du mouvement est trop faible, on va passer a 1.5 secondes.

Amélioration 8 - Ajout d'une barre de progression

```
if frame_index % 100 == 0:
    progress = frame_index / frame_count * 100
    print(f"Progression : {progress:.1f}%")
```

# Objectifs à venir

À ce stade du projet, les principales fonctionnalités de base ont été mises en place. Le programme est désormais capable d'analyser une vidéo, de mesurer le niveau de mouvement entre les images, de détecter les moments les plus dynamiques et de générer automatiquement une vidéo « best-of » contenant les passages jugés les plus intéressants. Des améliorations ont également été apportées afin de rendre l'outil plus robuste et plus utilisable, notamment avec l'ajout d'une barre de progression lors de l'analyse et la limitation de la durée du best-of afin d'éviter la génération de vidéos trop longues.

Cependant, plusieurs axes d'amélioration restent à explorer afin d'augmenter la pertinence et les performances du système.

Tout d'abord, une amélioration importante concerne la qualité de la détection des moments forts. Actuellement, le programme se base principalement sur l'intensité du mouvement entre les images pour identifier les segments intéressants. Dans les prochaines étapes du projet, l'objectif sera d'affiner ce mécanisme en introduisant des méthodes supplémentaires d'analyse vidéo. Par exemple, la détection plus précise des transitions de scène permettra d'identifier les changements de plan, ce qui constitue souvent un indicateur important dans les montages vidéo.

Un autre objectif consiste à intégrer l'analyse audio de la vidéo. En effet, certains moments marquants d'une vidéo ne sont pas uniquement caractérisés par le mouvement, mais également par le son, comme des cris, des explosions ou des applaudissements. L'ajout

d'un module d'analyse audio permettra donc d'identifier les pics d'intensité sonore et de les combiner avec les informations visuelles afin d'améliorer la détection des moments clés.

Dans un second temps, ces deux types d'informations — visuelles et sonores — pourront être fusionnés dans un système de score global. Chaque segment de la vidéo pourra alors être évalué selon plusieurs critères, comme le niveau de mouvement, la présence d'un changement de scène et l'intensité sonore. Cette approche multimodale permettra d'obtenir une sélection de moments forts plus pertinente et plus proche de ce que pourrait produire un montage humain.

Enfin, d'autres améliorations techniques pourront être envisagées, notamment l'optimisation des performances du programme pour traiter des vidéos plus longues, ainsi que l'amélioration de la visualisation des résultats, par exemple en affichant directement sur le graphique les segments sélectionnés pour le best-of.

L'objectif final du projet est ainsi de développer un outil d'analyse vidéo capable de générer automatiquement des extraits pertinents à partir d'une vidéo complète, en s'appuyant sur différentes sources d'information et sur des méthodes issues du domaine de la vision par ordinateur et du traitement du signal.

# Lien de mon code source :

https://github.com/Cyriacccc/P2I.git

commande pour lancer le programme dans la console : python -m app.cli